RUN: OOP Unit 1, Classes and Objects

Nov 5, 2026 · @shahan

All of Unit 1, with every recent paper's questions solved in place. Allow about 2 hours, and keep a compiler open if you're feeling honest. **Every program here was compiled and run with g++**, so the code works and the outputs are real.

How this run works

Read, try the **PYQ** on paper, then compare. This paper rewards writing code by hand, so skim-reading it will feel like understanding right up until the hall.

What the paper likes: a full C++ program for 4 marks, usually with a scenario attached, and short theory on `this`, inline, and constructors. Scenarios change every year, but the skeleton doesn't.

Videos: Neso Academy C++ playlist for basics, and CppNuts C++ playlist for Class and Object, Constructors, Static Members and Inline Functions. Slides are in `OOP/Notes/Unit 1 slides.pdf`.

Stop 1: Why objects exist

Procedural code is a list of instructions passing data around like a shared Google Doc: everyone can edit everything, and something's always mysteriously broken. OOP bundles data with the functions allowed to touch it.

> **PYQ, Sessional-1 2026 Q1(a), this year's paper.** What is the Object-Oriented Programming paradigm? Explain its key features with examples.

Answer: OOP organises a program around objects, which combine data and the functions that work on it. Its key features:

- **Encapsulation:** data and functions are bundled, and data is hidden. A `BankAccount`'s balance is private.
- **Abstraction:** only what's essential is exposed. You call `withdraw()` without knowing its internals.
- **Inheritance:** a new class reuses an existing one. `Car` inherits from `Vehicle`.
- **Polymorphism:** one name, many forms. `area()` behaves differently for `Circle` and `Square`.

OOP builds a program out of objects that bundle data with the functions allowed to touch it, resting on encapsulation, abstraction, inheritance and polymorphism.

> **PYQ, Sessional-1 2025 Q1(a), 2 marks.** List 5 key differences between procedure-oriented and object-oriented programming.

Answer:

Basis	Procedure-oriented	Object-oriented
Focus	Functions	Objects and data
Approach	Top-down	Bottom-up
Data	Global, moves freely	Hidden in objects
Security	Low	Access specifiers protect data
Reuse	Limited	Inheritance and templates
Example	C	C++, Java

Procedural code is organised around functions, and OOP around objects that guard their own data.

Stop 2: Access specifiers

> **PYQ, Endsem 2025 Q1(a), 4.5 marks.** How do private, public and protected control data hiding and accessibility? Explain with an example showing access from within and outside a class.

Answer:

Specifier	Inside class	Derived class	Outside
private	Yes	No	No
protected	Yes	Yes	No
public	Yes	Yes	Yes

```
class A {
    int pri = 1;       // private by default
protected:
    int pro = 2;
public:
    int pub = 3;
    int sum() { return pri + pro + pub; }   // all allowed inside
};
class B : public A {
    int f() { return pro + pub; }           // pri is not allowed here
};
int main() {
    A a;
    a.pub = 5;         // fine
    // a.pri = 5;      // error: private
    // a.pro = 5;      // error: protected
}
```

Private data plus public methods is what data hiding means: the outside world can only go through the methods, so it can't put the object in a nonsense state.

private: the class only. protected: the class and its derived classes. public: everyone.

Stop 3: The small constructs

> **PYQ, Sessional-1 2025 Q1(b), 2 marks.** Explain with a code snippet: this pointer, default arguments, function overloading, virtual function.

Answer:

```
class Box {
    int w;
public:
    void setW(int w) { this->w = w; }         // this: pointer to the current object
    int area(int h = 1) { return w * h; }     // default argument
    int scale(int k) { return w * k; }        // overloading: same name...
    double scale(double k) { return w * k; }  // ...different parameters
    virtual void show() { cout << "Box"; }    // virtual: chosen at run time by object type
};
```

`this` points to the calling object. Default arguments fill in missing arguments. Overloading means same name, different parameters. A virtual function is chosen by the object's type at run time.

> **PYQ, Endsem 2025 Q1(c), 4.5 marks.** Explain with code: (i) inline function, (ii) dynamic constructor, (iii) return by reference.

Answer:

```
#include <iostream>
#include <cstring>
using namespace std;

// Inline function: the compiler may paste the body at the call site
inline int square(int x) { return x * x; }

// Dynamic constructor: allocates memory with new at creation time
class Name {
    char* s;
public:
    Name(const char* t) { s = new char[strlen(t) + 1]; strcpy(s, t); }
    ~Name() { delete[] s; }
    void show() const { cout << s << "\n"; }
};

// Return by reference: returns the variable itself, so the call can be assigned to
int& bigger(int& a, int& b) { return a > b ? a : b; }

int main() {
    cout << square(5) << "\n";
    Name n("Jamia");
    n.show();
    int x = 3, y = 7;
    bigger(x, y) = 100;      // changes y
    cout << x << " " << y << "\n";
}
```

Output: `25`, `Jamia`, `3 100`.

The warning they love: never return a reference to a local variable. It's gone when the function ends, and the reference points at a crime scene.

Inline saves the call overhead, a dynamic constructor allocates memory with new, and return by reference lets the call sit on the left of =.

> **PYQ, Sessional-1 2023 Q1(b), 1 mark.** Explain the uses of `const` in `Point midPoint(const Point & p) const;`

Answer: `const Point & p` means the function won't change the argument, and passing by reference avoids a copy. The final `const` means it won't change the object it's called on, so it can be called on const objects.

The first const protects the argument, and the last const protects the calling object.

Stop 4: Constructors and static members

Constructors run automatically at creation, and have no return type. There's default, parameterized and copy. Destructors run at the end, in reverse order, and exist mostly to return what you took.

Constructor	Signature	Runs when
Default	`Point()`	`Point a;`
Parameterized	`Point(double x, double y)`	`Point b(3, 4);`
Copy	`Point(const Point& p)`	`Point c = b;` or passing by value
Destructor	`~Point()`	The object goes out of scope or is deleted

Copy constructor: `Point(const Point& p)`. It takes a reference, because taking it by value would need a copy, which needs a copy constructor, which needs a copy, and so on until the stack gives up.

Static members belong to the class, and there's one shared copy. Define the variable outside the class, once.

> **PYQ, Sessional-1 2023 Q1(c), 3 marks.** Define a class Point with two double attributes `_x` and `_y`. Provide default, parameterized and copy constructors, a default destructor, and a count variable tracking how many objects are in the program.

Answer:

```
#include <iostream>
using namespace std;

class Point {
    double _x, _y;
    static int count;
public:
    Point() : _x(0), _y(0) { count++; }
    Point(double x, double y) : _x(x), _y(y) { count++; }
    Point(const Point& p) : _x(p._x), _y(p._y) { count++; }
    ~Point() { count--; }
    static int getCount() { return count; }
};
int Point::count = 0;

int main() {
    Point a, b(1, 2), c(b);
    cout << Point::getCount() << "\n";
    {
        Point d;
        cout << Point::getCount() << "\n";
    }
    cout << Point::getCount() << "\n";
}
```

Output: `3`, `4`, `3`. The copy constructor must also count, or `c` goes undetected, and the destructor must subtract, or the count only ever goes up.

Make count a static member: increment it in every constructor, the copy constructor included, and decrement it in the destructor.

> **PYQ, Sessional-1 2026 Q2, this year's paper, 4 marks.** Design a class BedAllocation for hospital billing. Use constructor overloading to create records by ward type: General (Rs 500/day), Semi-Private (Rs 1500/day), Private (Rs 3000/day), ICU (Rs 8000/day + Rs 2000 fixed), with charges calculated automatically. Add display(), and use an array of objects for four patients in the four ward types.

Answer:

```
#include <iostream>
#include <string>
using namespace std;

class BedAllocation {
    string name, ward;
    int days;
    double charges;
public:
    // General Ward: name and days only
    BedAllocation(string n, int d) : name(n), ward("General"), days(d), charges(500.0 * d) {}
    // Semi-Private or Private: rate passed in
    BedAllocation(string n, int d, string w, double rate) : name(n), ward(w), days(d), charges(rate
* d) {}
    // ICU: rate plus a fixed charge
    BedAllocation(string n, int d, double rate, double fixedCharge)
        : name(n), ward("ICU"), days(d), charges(rate * d + fixedCharge) {}
    void display() const {
        cout << name << " | " << ward << " | " << days << " days | Rs " << charges << "\n";
    }
};

int main() {
    BedAllocation p[4] = {
        BedAllocation("Asha", 3),
        BedAllocation("Ravi", 2, "Semi-Private", 1500),
        BedAllocation("Sana", 2, "Private", 3000),
        BedAllocation("Imran", 2, 8000.0, 2000.0)
    };
    for (int i = 0; i < 4; i++) p[i].display();
}
```

Output: Rs 1500, Rs 3000, Rs 6000, Rs 18000. The overloads differ by parameter types, which is how the compiler picks one. There's no default constructor here, so the array has to be initialised explicitly.

Three overloaded constructors (General; Semi-Private or Private with a rate; ICU with a rate plus a fixed charge) compute the charges at creation. ICU for 2 days is 8000 × 2 + 2000 = Rs 18000.

Stop 5: Bigger class programs

> **PYQ, Endsem 2025 Q1(b), 4.5 marks.** Design a class Order for a restaurant with order id, table number, items, amount and status (Pending/Preparing/Served/Billed/Cancelled). Members: placeOrder() (default status Pending), updateStatus() (allowed values only), generateBill(). Add a menu-driven interface.

Answer:

```
#include <iostream>
#include <string>
using namespace std;

class Order {
    int orderId, tableNo;
    string items, status;
    double amount;
public:
    void placeOrder() {
        cout << "Order id, table no, amount: ";
        cin >> orderId >> tableNo >> amount;
        cin.ignore();
        cout << "Items: ";
        getline(cin, items);
        status = "Pending";
    }
    void updateStatus() {
        string s;
        cout << "New status: ";
        cin >> s;
        if (s == "Pending" || s == "Preparing" || s == "Served" || s == "Billed" || s ==
"Cancelled")
            status = s;
        else
            cout << "Invalid status\n";
    }
    void generateBill() const {
        cout << "Order " << orderId << ", Table " << tableNo << ", Items: " << items
             << ", Total: " << amount << ", Status: " << status << "\n";
    }
};

int main() {
    Order o;
    int ch;
    do {
        cout << "1.Place 2.Update 3.Bill 0.Exit: ";
        cin >> ch;
        switch (ch) {
            case 1: o.placeOrder(); break;
            case 2: o.updateStatus(); break;
            case 3: o.generateBill(); break;
        }
    } while (ch != 0);
}
```

`cin.ignore()` matters: without it, `getline` eats the leftover newline and your items come out blank. It's the classic silent bug.

updateStatus() must reject anything outside the five allowed values. That check is where the marks for this question are.

> **PYQ, Sessional-1 2024 Q1, 5 marks.** Model students and courses using classes Student and Course. Students can enroll and withdraw. Each course has a maximum capacity, keeps track of enrolled students, and handles adding and removing by ID.

Answer:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
    int id;
    string name;
    Student(int i = 0, string n = "") : id(i), name(n) {}
};

class Course {
    string title;
    int capacity, count;
    Student* enrolled;
public:
    Course(string t, int cap) : title(t), capacity(cap), count(0) { enrolled = new Student[cap]; }
    ~Course() { delete[] enrolled; }
    bool enroll(const Student& s) {
        if (count == capacity) { cout << "Course full\n"; return false; }
        enrolled[count++] = s;
        return true;
    }
    bool withdraw(int id) {
        for (int i = 0; i < count; i++)
            if (enrolled[i].id == id) {
                enrolled[i] = enrolled[--count];   // move the last one into the gap
                return true;
            }
        return false;
    }
    void show() const {
        cout << title << ":";
        for (int i = 0; i < count; i++) cout << " " << enrolled[i].name;
        cout << "\n";
    }
};

int main() {
    Student a(1, "Asha"), b(2, "Ravi"), c(3, "Sana");
    Course oop("OOP", 2);
    oop.enroll(a); oop.enroll(b); oop.enroll(c);
    oop.show();
    oop.withdraw(1); oop.enroll(c);
    oop.show();
}
```

Output: `Course full`, `OOP: Asha Ravi`, `OOP: Ravi Sana`. The default arguments on Student's constructor are what allow `new Student[cap]`.

Course holds an array of Students plus a count. enroll() checks capacity first, and withdraw() finds the student by ID and moves the last student into the gap.

> **PYQ, Sessional-1 2022 Q3, 4 marks.** Explain how an object can be used as a function argument, with a program that adds two time objects in hour:minute format.

Answer: An object is passed like any variable: by value, which makes a copy, or by reference. The called function can access its private members because it's a member of the same class.

```
#include <iostream>
using namespace std;

class Time {
    int h, m;
public:
    Time(int a = 0, int b = 0) : h(a), m(b) {}
    Time add(Time t) const {             // object as a function argument
        Time r;
        r.m = m + t.m;
        r.h = h + t.h + r.m / 60;
        r.m %= 60;
        return r;
    }
    void show() const { cout << h << ":" << m << "\n"; }
};
int main() {
    Time a(2, 45), b(1, 30);
    a.add(b).show();
}
```

Output: `4:15`, since 2:45 + 1:30 = 3:75, and the 75 minutes carry over to make 4:15.

`a.add(b)` passes b by value. Carry the minutes with `/ 60` and `% 60`.

Stop 6: Friend functions

A friend isn't a member, but it can access private data. Declare it inside the class with `friend`, and define it outside like a normal function. It's needed when a function must see inside two classes at once. Friendship is granted, never taken, and it isn't inherited, which is more boundary-setting than most people manage.

> **PYQ, Sessional-1 2025 Q2, 4 marks.** Create classes TemperatureSensor (private reading in °C) and HumiditySensor (private reading in %). Prompt for the number of sensors, collect readings and display them. Define a friend function computeAverage() that directly accesses both classes' private members to display the averages.

Answer:

```
#include <iostream>
using namespace std;

class HumiditySensor;   // forward declaration, so the friend can name it

class TemperatureSensor {
    float temp;
public:
    void read() { cout << "Temperature (C): "; cin >> temp; }
    void show() const { cout << "Temp: " << temp << " C\n"; }
    friend void computeAverage(TemperatureSensor[], HumiditySensor[], int);
};

class HumiditySensor {
    float hum;
public:
    void read() { cout << "Humidity (%): "; cin >> hum; }
    void show() const { cout << "Humidity: " << hum << " %\n"; }
    friend void computeAverage(TemperatureSensor[], HumiditySensor[], int);
};

void computeAverage(TemperatureSensor t[], HumiditySensor h[], int n) {
    float st = 0, sh = 0;
    for (int i = 0; i < n; i++) { st += t[i].temp; sh += h[i].hum; }
    cout << "Avg temp: " << st / n << ", Avg humidity: " << sh / n << "\n";
}

int main() {
    int n;
    cout << "Number of sensors: "; cin >> n;
    TemperatureSensor* t = new TemperatureSensor[n];
    HumiditySensor* h = new HumiditySensor[n];
    for (int i = 0; i < n; i++) { t[i].read(); h[i].read(); }
    for (int i = 0; i < n; i++) { t[i].show(); h[i].show(); }
    computeAverage(t, h, n);
    delete[] t; delete[] h;
}
```

The forward declaration on the third line is what the examiner checks for.

Declare computeAverage as a friend inside both classes, with a forward declaration of the second class, so it can read both private readings.

> **PYQ, Sessional-1 2022 Q4, 3 marks.** Why is a friend function needed? Write a program for the distance between two points using inline and friend functions.

Answer: It's needed when a non-member must reach private data, such as when working across two objects or classes.

```
#include <iostream>
#include <cmath>
using namespace std;

class Point {
    float x, y;
public:
    Point(float a, float b) : x(a), y(b) {}
    friend float distanceBetween(const Point&, const Point&);
};
inline float distanceBetween(const Point& p, const Point& q) {
    return sqrt((p.x - q.x) * (p.x - q.x) + (p.y - q.y) * (p.y - q.y));
}
int main() {
    Point a(0, 0), b(3, 4);
    cout << distanceBetween(a, b) << "\n";
}
```

Output: `5`.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

A friend function reaches private data across objects without being a member. Here it reads both points' coordinates directly.

That's Unit 1. Next is Unit 2, where the operators learn your classes' names.